

TECHNICAL DOCUMENT

Capacity Planning Overview

Business Central Control Add-in — Architecture, Data Flow & DHTMLX Integration

Extension: DailyOptimizer (Optimizers) — app id range 50600–60700

Page: 50722 "Capacity Planning Overview" | Controladdin: DHXCapacityPlanningOverviewAddin

Codeunit: 50604 "DHX Data Handler" — region CPO_

Reference prototype: KLAAS/CPO_v131 (DHTMLXtempv112-app.js / DHTMLXtempv112-data.js)

Folder: src\dhx\capacity_planning_overview\

Document date: 2026-09-15 (supersedes the 2026-09-04 revision)

Revision note (2026-09-15)

This revision documents a same-day follow-up session on top of the 2026-09-04 confirm-gated-reschedule work already described throughout this document: (1) Section 3's own "Requested" demand side (§7) was **flipped from INCLUDE-only (the inspected Work Order's own demand) to EXCLUDE (company-wide, minus the inspected Work Order)** — matching Section 4's own long-standing direction, so dragging a bar to relocate work now shows genuine competing demand from everyone else that day; the "Show Data" drill-down's own job/task include-filter was removed to match, since it would otherwise open the exact wrong record set once the segment itself excludes that job/task; (2) **Section 1's shortage/coverage engine (§9) was replaced outright**, not just re-scoped — the former before/after MARGINAL-delta formula (evaluateWO/currentPositionShortage) is gone, replaced by a new excludingWOFlow() that reads the max-flow result of the SAME company-wide-minus-this-WO baseline DIRECTLY, with nothing added back in; a direct, intended consequence is that Section 1 is now **position-independent** — dragging a bar in Section 2 no longer changes Section 1's own numbers at all; (3) a new title-bar **audit-totals box** (§4.1) gives the user a manual "Requested: X hours | Assigned: Y hours" cross-check total for the inspected Work Order, for eyeballing against Section 3's per-day bars, sharing a new computeWorkOrderTotals() helper with Section 2's own rendering so the two can never drift apart; (4) a critical verification-methodology finding (§13.12): this add-in's background-loaded "other Work Order" data can take well over a minute to fully arrive, and a same-day live check that didn't wait for it undercounted a real total by **62×** (376h measured vs. the true 43,240h) despite passing an internal Section-3-vs-Section-4 self-consistency check — any future verification pass on this add-in **MUST** poll #cpo-bg-loading before trusting a total; (5) a second, equally important finding (§13.13), surfaced while building item (3) above: Section 2's own rendered chips are a client-side, window-relative "reschedule preview" of workOrderSequences[], NOT a literal render of dayPlanningLines[]'s stored Plan Date — this exact confusion produced two consecutive wrong title-box implementations before the correct, shared-array design landed. Every changed section below is marked **(2026-09-15)** inline; everything else, including the 2026-09-04 revision's own marked changes, is unchanged from the prior revision.

Table of Contents

1. Overview & Purpose
2. Architecture — BOOT / Wrapper / Controladdin Pattern

- 2.1 File Inventory
- 2.2 Controladdin Contract

3. End-to-End Data Flow

- 3.1 Confirm-Gated Reschedule (2026-09-04)
- 3.2 Processing Indicators (2026-09-04)

4. The JSON Payload Contract (window.DHTMLXPlannerData shape)

- 4.1 Title-Bar Audit-Totals Box (NEW, 2026-09-15)

5. Section 1 — Stats Header

6. Section 2 — Work Order's Own Day Planning Scheduler

7. Section 3 — Capacity vs Requested Daily Bars

8. Section 4 — Skill → Job/Task → Sequence Tree

9. The Shortage / Coverage Engine (Max-Flow)

10. AL-Side Implementation Details

11. Cross-Work-Order Scoping (2026-09-03 Correction)

12. Performance — Page Background Task Pagination

13. Known Pitfalls, Bugs Found & Fixes Applied

14. Appendix — File Reference

1. Overview & Purpose

The Capacity Planning Overview add-in is a Business Central control add-in that gives a single-screen view, for one selected Work Order, of: (a) whether that Work Order's own demand can realistically be covered by the company's resource pool day-by-day, (b) the Work Order's own Day Planning schedule, (c) a daily capacity-vs-request bar chart, and (d) a tree of every other Work Order's competing demand in the same visible window.

It is a faithful, BC-native port of a standalone DHTMLX prototype (KLAAS\CP0_v131\DHTMLXtempv112-app.js), with two deliberate substitutions: (1) the prototype's trial/evaluation @dhx/trial-scheduler npm package is replaced by this repository's already-licensed dhtmlxscheduler.js, and (2) all data is now real Business Central records instead of a static mock JSON file. The prototype's own client-side algorithms (shortage/coverage max-flow engine, tree builder, daily bar aggregator) are ported near-verbatim — Business Central's job is reduced to producing one JSON payload shaped exactly like the prototype's own mock window.DHTMLXPlannerData object.

Guiding principle (explicit, non-negotiable project requirement)

A BC control add-in binds exactly one JS component via one wrapper.js. All four visual sections, the shared header, and the "Days to show" control therefore live inside ONE JS class (CapacityPlanningOverview), not four separate usercontrol blocks. AL/Business Central is treated purely as a data source and event sink — "consider BC Add-ins as a browser for JS". The **2026-09-04 confirm-gated redesign (§3.1) sharpens this further**: AL is now not even an event sink for every interaction — it is a data source on load, and an event sink only for the one action the user deliberately commits to.

2. Architecture — BOOT / Wrapper / Controladdin Pattern

This follows the same skeleton every other DHTMLX add-in in this repository uses (e.g. projectschedule, request_assignment):

- BC loads the controladdin's StartupScript, which calls `BOOT()`.
- `BOOT()` (in wrapper.js) finds the host `<div id="controlAddIn">`, normalizes it to fill 100% width/height, renames its id to `cpo-root`, instantiates new `CapacityPlanningOverview('cpo-root')` (stored at `window.__cpo`), shows the full-page loading overlay (§3.2), and calls `Microsoft.Dynamics.NAV.InvokeExtensibilityMethod("ControlReady", [])`.
- AL's `ControlReady` trigger (page 50722) fires, which builds the initial JSON payload and calls `CurrPage.DhxCpo.SetPlanningData(json)`.
- `SetPlanningData` (the controladdin procedure) invokes the JS-side global function of the same name (in wrapper.js), which parses the JSON and calls `window.__cpo.applyPlanningData(json)`.
- `applyPlanningData` is the single entry point that fans out to every section's own `render*` method, then hides the loading overlay as its true last step (§3.2) — the user's actual "processing is done, ready for the next action" signal.

2.1 File Inventory

File	Role
DHXCcapacityPlanningOverviewAddin. ControlAddin.al	Controladdin declaration: script/stylessheet load order, procedures (AL→JS), events (JS→AL), sizing properties. Unchanged in the 2026-09-04 session — the confirm-gated redesign changed WHEN the JS side calls the existing <code>OnRescheduleWorkOrder</code> event, not the event's own signature.

File	Role
page_50722_CapacityPlanningOverview.al	Card page hosting the single usercontrol. No SourceTable. Holds GWorkOrderNo / DaysToShow state, rebuilds and pushes the payload via RefreshData(). OnRescheduleWorkOrder's stub Message() dialog was removed (2026-09-04, §13.10) — it is now a true silent no-op stub, matching OnRequestCapacityLookup/OnSequenceChipClick.
startupScript.js	One line: BOOT();
wrapper.js	Thin BOOT/binding shell — instantiates the JS class, exposes the AL-facing globals (SetPlanningData / SetColors / LoadCapacityLookup / AppendOtherWorkOrderData), and (2026-09-04) owns the full-page loading overlay's safety timer (§3.2).
capacityPlanningOverview.js	The single JS component (~1,850 lines). All four sections, the shared header, the shortage/coverage engine, scroll-sync, tooltips, the processing-indicator methods, and (2026-09-04) the Confirm-button / unconfirmed-change state machine.
style.css	All layout/heat-map/chip/scrollbar CSS for the four sections and shared header, plus (2026-09-04) the spinner/overlay and Confirm-button CSS.
codeunit_50604_DHXDataHandler.al	Shared data-handler codeunit (also used by other DHX add-ins). Region CPO_ builds the one combined JSON payload.
Pag50662.WorkorderCard.al	Launching page — action CapacityPlanningOverviewAct calls SetWorkOrderNo() then Run() (top-level navigation, not a modal dialog).

Vendor libraries (dhtmlxscheduler.js/.css, suite.js/.css, GlobalFunction.js) are shared, root-level copies reused by every DHX add-in — no per-folder vendor duplication.

2.2 Controladdin Contract

```

controladdin DHXCapacityPlanningOverviewAddin
{
    RequestedHeight = 1150; MinimumHeight = 420;    VerticalShrink/Stretch = true;
    RequestedWidth = 1800; MinimumWidth = 900;      HorizontalShrink/Stretch = true;

    Scripts = dhtmlxscheduler.js, suite.js, GlobalFunction.js,
              capacityPlanningOverview.js,    // component class BEFORE wrapper.js
              wrapper.js;                     // BOOT() references the class - must load last

    StartupScript = startupScript.js;
    StyleSheets = dhtmlxscheduler.css, suite.css, style.css;

    procedure SetPlanningData(PlanningDataJsonTxt: Text);
    procedure SetColors(ColorsJsonTxt: Text);
    procedure LoadCapacityLookup(CapacityLookupJsonTxt: Text);
    procedure AppendOtherWorkOrderData(OtherWorkOrderDataJsonTxt: Text);
    procedure NotifyOtherWorkOrderDataTaskPending();
    procedure StopOtherWorkOrderDataPolling();

    event ControlReady();
    event OnRescheduleWorkOrder(DayShift: Integer; PayloadJsonTxt: Text); // now fired ONLY
by Confirm - see 3.1
    event OnRequestCapacityLookup(FilterJsonTxt: Text);
    event OnSequenceChipClick(PayloadJsonTxt: Text);
    event OnDaysToShowChanged(NumberOfDays: Integer);
    event OnPollOtherWorkOrderDataResult();
}

```

RequestedHeight = 1150 is sized to the worst case: section 1 (~104px fixed) + section 2 (capped 400px) + section 3 (~180px) + section 4 (capped 420px) + ~40px chrome. MinimumHeight = 420 with VerticalShrink = true lets a small Work Order shrink the control naturally instead of leaving dead space.

3. End-to-End Data Flow

```

Workorder Card (page 50662)
  action CapacityPlanningOverviewAct
    CPO.SetWorkOrderNo(Rec."Work Order No.")
    CPO.Run()                                plain top-level nav, NOT RunModal()

page 50722 "Capacity Planning Overview"
  trigger ControlReady()
    EnsureDaysToShow()                      DaysToShow default = 30
    RefreshData()

  local procedure RefreshData()
    PlanningDataJson := DHXDataHandler.CPO_BuildPlanningDataJson_Paged(...)
    CurrPage.DhxCpo.SetPlanningData(PlanningDataJson)
                                                    (controladdin procedure -> JS global fn, same name)

wrapper.js
  window.SetPlanningData(json) { window.__cpo.applyPlanningData(JSON.parse(json)); }

capacityPlanningOverview.js
  applyPlanningData(json)
    this.db = json; this.dates = [...]; this.woAnchor = 0; this._hasUnconfirmedChanges = false
    this._baseRequests = this.aggregateRequests()
    this._baselineWithoutWO = this.aggregateOutstandingRequestsWithoutWO()
    renderWoSummaryScheduler(db)              Section 1
    renderWoScheduler(db)                    Section 2 (ends by calling renderWorkOrder(), which also
                                                    does a first-pass render of Section 3)
    measuredColumnWidth = measureColumnWidth() needs a REAL rendered cell from Section 2
    renderCapacityBars(db)                   Section 3 (re-rendered with the correct measured width)
    renderCentralTree(db)                   Section 4
    bindScrollSync()                        wires the shared horizontal scrollbar to all 4 sections
    updateConfirmButtonState()              resets the Confirm button to disabled - fresh server data
    hideLoading()                          true last step - see 3.2

```

Round-trip back to AL happens on three occasions (updated 2026-09-04 — was previously two immediate occasions plus a stub; is now genuinely gated on the two data-changing ones):

- "Days to show" changed (JS-owned input, top bar) → OnDaysToShowChanged(NumberOfDays) → AL sets DaysToShow and calls RefreshData() again (full round-trip, new payload) — unchanged; this genuinely needs new data the client doesn't have.
- **The Confirm button** (new, 2026-09-04) → OnRescheduleWorkOrder(woAnchor, payloadJson) — the ONE deliberate point any reschedule work (Section 2 drag, Section 3 click-to-relocate) reaches AL at all. Still a stub server-side (the real transactional Validate("Plan Date", ...) loop is a documented later step) — see §3.1.
- Section 4 chip click → OnSequenceChipClick(payloadJson) — stub, intended to open "Day Plannings" filtered to that specific line. Left as an immediate call (explicit 2026-09-04 decision, §3.1) — it is a navigate/view

action, not a schedule edit, so it is not part of the confirm-gated flow.

Section 3's Expand / Exp. to Task / Collapse buttons, the Capacity Lookup double-click, Section 2's own drag, and Section 3's click-to-relocate are all pure client-side, no AL round-trip per action — see §3.1 for the two that changed this session.

3.1 Confirm-Gated Reschedule (2026-09-04)

Prior behavior: every single Section 2 drag AND every Section 3 click-to-relocate immediately called `InvokeExtensibilityMethod('OnRescheduleWorkOrder', [shift, json])` — one real BC round-trip per intermediate mouse action, even though the trigger was (and still is) a complete no-op stub, and even though every value that call needed was already known client-side before it fired.

User-driven redesign

Stated directly by the user across a series of messages in the same session: "as long as no confirm data from JS then you must use in JS calculation/process... add confirm button in same row of title... if user clicked that button then you start round-trip with BC to update data"; confirmed again as "all action in JS must calculated in JS, do not act round-trip with BC" and "so no more round-trip with bc as long as no clicked on confirmed button". Final shape, confirmed against all four sections explicitly: Section 1 is a calculated board that always runs in JS; Section 2 is the Work Order to be modified (shiftable left/right), no round-trip until Confirm; Section 3 is user-driven and drives Section 2, also no round-trip until Confirm; Section 4 is static existing-data context (excluding this Work Order's own lines — §11) and is never touched by reschedule at all.

New client-side state machine (all in `capacityPlanningOverview.js`):

Method	Role
<code>moveWorkOrderToDay(dayIndex)</code>	Section 3 click-to-relocate. Updates <code>woAnchor</code> , resets anchor-dependent caches, calls <code>renderWorkOrder()</code> (Sections 1-3 re-render), then <code>markUnconfirmedChange()</code> . No AL call.
<code>onEventChanged (Section 2 drag handler)</code>	Same shape — computes the day-shift from the drop position, updates <code>woAnchor</code> if valid, re-renders, calls <code>markUnconfirmedChange()</code> . No AL call.
<code>markUnconfirmedChange()</code>	Sets this._hasUnconfirmedChanges = true and calls <code>updateConfirmButtonState()</code> — the only side effect of any local move.
<code>updateConfirmButtonState()</code>	Enables/disables <code>#cpo-confirm-btn</code> and toggles its <code>.cpo-confirm-btn-pending</code> CSS class (muted/disabled → solid blue "click me" look) to match the current flag.
<code>bindConfirmButton()</code>	Wires the button's click handler — calls <code>confirmChanges()</code> only if there is actually something unconfirmed.
<code>confirmChanges()</code>	The ONE deliberate BC round-trip point. <code>showLoading()</code> , <code>InvokeExtensibilityMethod('OnRescheduleWorkOrder', [this.woAnchor, json])</code> with the CURRENT (possibly several-moves-later) <code>woAnchor</code> sent ONCE, then clears the pending flag, updates the button, <code>hideLoading()</code> .

`#cpo-confirm-btn` ("Confirm changes") lives in the top bar, same row as the title, right of `#cpo-title` and left of the "Days to show" input — per the user's explicit placement request. It starts disabled/muted on every fresh load (`applyPlanningData` resets `_hasUnconfirmedChanges` to false and calls `updateConfirmButtonState()`), since a fresh server payload has nothing unconfirmed by definition.

Section 1 and Section 4, confirmed unaffected: Section 1 was already pure JS (`renderWorkOrder()` calls `woSummaryScheduler.setCurrentView(...)` to re-read the current `woAnchor`/caches — no AL involvement, before or after this redesign). Section 4 was already never re-rendered by any reschedule path (`renderWorkOrder()` does not call `renderCentralTree()`) — both facts predate this session and are unchanged by it; this redesign only removed the AL call that used to sit at the END of the Section 2/3 render paths.

3.2 Processing Indicators (2026-09-04)

New UI feedback for every action that genuinely takes time, added on explicit request ("the data load and processing take a time... show processing animated"). Both use the same visual language: a ring spinner (`.cpo-spinner`, a conic-gradient dark arc on a light track, `CSS @keyframes cpo-spin`), at two sizes/contexts.

Indicator	Shown for	Hidden by
<code>#cpo-loading-overlay</code> (full-host, blocking, <code>.cpo-loading-overlay</code>)	Initial ControlReady round-trip; a "Days to show" reload; Expand/Collapse-all's DHTMLX rebuild (via <code>runBusy</code>).	<code>applyPlanningData</code> 's last step (<code>hideLoading()</code>); a 3-minute safety timer guards against it ever sticking.
<code>#cpo-bg-loading</code> ("Loading more data...", top bar, <code>.cpo-inline-loading</code> , non-blocking)	The background other-Work-Order-data pagination poll (§12) while it is still running.	<code>StopOtherWorkOrderDataPolling</code> (a result arrived) or the poll's own 30s ceiling.

`runBusy(fn)` exists specifically for GENUINELY slow (multi-second) synchronous CPU work with no natural browser yield point — today, only `setTreeOpenState`'s Expand/Collapse-all rebuild qualifies. It shows the overlay, waits two `requestAnimationFrame` callbacks (forcing one real paint before the heavy work runs — showing the overlay and running the work in the same tick would never let the browser paint it at all), runs the work, then hides. It is deliberately NOT used around `reschedule` (`moveWorkOrderToDay`/the drag handler) any more — see §13.9 for why plain `showLoading()/hideLoading()` around a fire-and-forget `InvokeExtensibilityMethod` call never needed this trick, and why `reschedule`'s own ~100-150ms local render cost is not worth a spinner flash for at all now that it has no BC call to wait on.

4. The JSON Payload Contract

Built entirely by `codeunit 50604.CPO_BuildPlanningDataJson_Paged(WorkOrderNo, NumberOfDays, MaxOtherLines, out RemainingGroupKeys)`, shaped to mirror the reference prototype's own mock `window.DHTMLXPlannerData` object field-for-field. This is the only data BC ever pushes into the JS component on load — every section's render function derives everything else (shortage %, chip layout, bar heights, tree grouping) from this one object client-side, and (2026-09-04) every subsequent user interaction short of `Confirm` derives its results from this same object too, never re-fetching it.

Field	Type	Meaning / Source
<code>daysToShow / startDate / endDate</code>	int / ISO date	Echoed back so the JS-owned input can sync to whatever AL actually used. <code>endDate = startDate + daysToShow - 1</code> .
<code>workdays[]</code>	ISO date[]	Every calendar day in range (weekends included, not skipped) — the reference's own convention. Also doubles as the implicit workday-offset anchor for Section 2's bars.
<code>workOrder.no / .description / .notEarlierThan / .notLaterThan</code>	obj	The inspected Work Order's own header. <code>description</code> feeds the JS title bar directly.
<code>project.no / .description</code>	obj	The Work Order's parent Job ("Project No.").
<code>skills[]</code>	array	One entry per distinct Skill demanded (union of the inspected WO's own skills + every other WO's, §11). Each: <code>code</code> , <code>color</code> , <code>textColor</code> , <code>border</code> , <code>dark</code> , <code>light</code> .
<code>resources[]</code>	array	Every resource holding any demanded skill, capped at 15/skill for max-flow performance. Each: <code>name</code> (= Resource No.), <code>skills[]</code> .

Field	Type	Meaning / Source
baseCapacity	number	Flat 8 (hours/resource/day).
externalFree[]	number[]	One entry per day in range — summed real "Res. Capacity Entry". Capacity for the capped resource pool with a non-blank Vendor No.
groups[]	array	Section 4's tree source. One entry per distinct Skill demanded by every OTHER Work Order in this window. Each has details[] = one entry per distinct Job No./Job Task No.
dayPlanningLines[]	array	Real Day Planning rows: the inspected WO's own lines plus every other WO's lines in the same window. Each carries a normalized workOrderNo.
workOrderSequences[]	array	Section 2's source. One entry per distinct Skill+Job+Task+SequenceNo belonging ONLY to the inspected Work Order, plus a per-(sequence, workday-offset) requested-hours map.

dayPlanningLines[] — per-line shape

```
{
  id, workOrderNo, job, task, description, sequenceLineNo,
  requestDate, requestedStartTime, requestedEndTime, requestedHours, requestedSkill,
  assignedResourceNo, assignedDate, assignedStartTime, assignedEndTime, assignedHours,
  sequenceNo
}
```

4.1 Title-Bar Audit-Totals Box (NEW, 2026-09-15)

A small bordered pill (#cpo-title-audit, class .cpo-title-audit-box) rendered immediately right of the title text, reading **"Requested: N hours | Assigned: N hours"**. Purpose, stated directly by the user: a manual cross-check total to compare against Section 3's own per-day bars while eyeballing whether a day's numbers reconcile — "it is for audit during matching with a day selection in section 3".

Computed entirely client-side (no new AL field, no extra round-trip) from `this._woEvents` — the LITERAL SAME array `renderWorkOrder()` (§6) just built and handed to DHTMLX for Section 2's own bars, stored on the instance for exactly this reuse. A shared method, `computeWorkOrderTotals()`, sums that array's own `requestedHours/assignedHours` fields (per-event "Assigned" is already `Math.round(requestedHours * assignment.pct / 100)`, computed once inside `renderWorkOrder()` itself, not re-derived here) — `renderTitleAuditBox()` then reads that shared total and updates the DOM. This is a deliberate reuse-over-duplication design, requested explicitly: "you must have a common function for that, because maybe used by other Add-ins" — no add-in elsewhere in this repository had an existing equivalent helper to reuse, so this one is new but shared within this class rather than inlined in two places.

`renderTitleAuditBox()` is called from the END of `renderWorkOrder()` itself — i.e. on EVERY reposition (the initial render via `renderWoScheduler`, a Section 2 drag, Section 3's click-to-relocate/`moveWorkOrderToDay`), not just the initial server payload, so the box stays live as the user interacts — matching its own stated audit purpose. `applyPlanningData()`'s own title-building code now only rebuilds the title TEXT span; the audit box is left entirely to `renderWorkOrder()/renderTitleAuditBox()`. Rendered ONLY when there is an inspected Work Order: guaranteed structurally by `renderWorkOrder()`'s own pre-existing `if (!this.woScheduler) return; guard` (page 50724/Dashboard's own `renderWoScheduler` override is a no-op and never sets `this.woScheduler` — see §8's own Dashboard notes) — no separate check was needed in the new code.

TWO consecutive wrong fixes, same day — the real lesson is in §13.13, read it before touching this again

Both of this box's first two implementations summed `db.dayPlanningLines[]` (first with no date check at all, then with a `dplDayIndex(line)-in-window` check added) — and BOTH were wrong, because Section 2's own chips are not built from that array's literal Plan Date at all. See §13.13 for the full story: what Section 2 actually renders from, why a ground-truth OData query on the real stored dates was checking the wrong thing entirely, and why any future total meant to match what Section 2 shows must read `this._woEvents` (via `computeWorkOrderTotals()`), never `db.dayPlanningLines[]/db.workOrderSequences[]` independently.

5. Section 1 — Stats Header

Function: `renderWoSummaryScheduler ( json )` | Host div: `#cpo-wo-summary` | View name: `workordersummary`

A 2-row synthetic `Scheduler.getSchedulerInstance()` timeline (NOT a real event calendar — it has no real events, only `cell_template`-rendered HTML per cell). The two rows are "Calculated conclusion" (% coverage) and "Current position shortage" (raw shortage hours) — **as of 2026-09-15, both rows read the SAME underlying number, formatted two ways** (see the algorithm change below); before that they were two functionally different marginal-delta figures.

createTimelineView option	Value	Note
<code>render</code>	<code>'bar'</code>	Standard timeline, not tree.
<code>x_unit / x_step / x_size</code>	<code>day / 1 / dates.length</code>	One column per visible day.
<code>y_unit</code>	2 synthetic sections	<code>{key:'fit',...}</code> , <code>{key:'shortage',...}</code> — not real DB rows.
<code>dy</code>	<code>CALC_ROW_HEIGHT = 34px</code>	Identical across all 3 Scheduler instances (1/2/4).
<code>column_width</code>	<code>COLUMN_WIDTH = 92px</code>	DHTMLX auto-stretches columns wider than this unless total content already exceeds container width.
<code>dx</code>	<code>ROW_LABEL_WIDTH = 360px</code>	Row-header width — the shared date-header row sections 1–3 all align under.
<code>scale_height</code>	<code>SCALE_HEIGHT = 36px</code>	
<code>cell_template</code>	<code>true</code>	Every cell's class AND value are custom HTML, computed live.

Data source: nothing from the payload directly — every cell calls `this.excludingWOFlow()[dayIndex]` LIVE at render time (§9), which reads `this.db.dayPlanningLines`, `this.db.resources`, `this.db.externalFree`. There is no AL-precomputed "statsRows" array. Unlike the pre-2026-09-15 design, `this.db.workOrderSequences` is NOT read here at all anymore — see below.

Algorithm replaced, not just re-scoped (2026-09-15, explicit user request)

Previously (§9's original design): `evaluateWO(dayIndex)/currentPositionShortage()` computed a BEFORE/AFTER MARGINAL delta — add this Work Order's own demand on top of everyone else's baseline, re-run the max-flow model, and report the INCREASE in shortage that addition caused ("how much worse do things get because of me"). The user asked for Section 1 to instead report the shortage/coverage of "all day planning in the window excluding this project task" DIRECTLY — not a delta. New method `excludingWOFlow()` reads `maxFlowDay()` against this `._baselineWithoutWO` (§9) straight, with nothing added back in. A direct, INTENDED consequence: Section 1 is now **position-independent** — dragging a bar in Section 2 no longer changes Section 1's own numbers at all, since a number that excludes this task's demand by definition cannot depend on where that excluded demand currently sits. `evaluateWO/currentPositionShortage/workOrderExtra(start)` were all removed (Section 2's own `currentPositionSkillShortage/workOrderExtraCurrent` are UNCHANGED — still a before/after delta, still used for the per-sequence-row shortage badges in §6, since Section 2 genuinely does still care where each occurrence currently sits).

`renderWorkOrder()` (called by both the drag handler and `moveWorkOrderToDay`) still calls `woSummaryScheduler.setCurrentView(...)` after every local move, matching the pre-2026-09-15 behavior — harmless now rather than load-bearing, since the cell templates just re-read the same cached, unchanged `excludingWOFlow()` result every time.

6. Section 2 — Work Order's Own Scheduler

Function: `renderWoScheduler(json) → renderWorkOrder()` | Host: `#cpo-wo-scheduler` (wrapped by `#cpo-wo-scheduler-wrap`, §13.5) | View: `workorder`

A real Scheduler timeline with native drag-move enabled. One row per distinct Skill+Job+Task+SequenceNo combination the inspected Work Order itself demands (from `workOrderSequences[]`). Each row's events are positioned via a `workday-offset` formula (`idxWork(woAnchor, offset)`), not raw calendar "Plan Date" placement — dragging a bar shifts a single client-side `woAnchor` integer and re-renders the whole section instantly (optimistic, pure client-side simulation matching the reference exactly).

createTimelineView option	Value	Note
render	'bar'	
y_unit	one section per sequence	Built from <code>json.workOrderSequences</code> ; falls back to a single "No Day Planning lines" placeholder row if empty.
dy / event_dy	32 / 24px	<code>ROW_HEIGHT / ROW_HEIGHT - 8</code> .
dx	360px	Same <code>ROW_LABEL_WIDTH</code> as section 1 (shares its date header).
scale_height	0	Deliberately zero — shares section 1's header row. DHTMLX's own empty header row is force-hidden via CSS.
config.drag_move	true	The only section with native drag.

Events (drag / click) — updated 2026-09-04:

Event	Behavior
onEventChanged (drag)	Computes the day-shift from the drop position, updates <code>woAnchor</code> if valid, resets anchor-dependent caches, re-renders (Sections 1-3), calls <code>markUnconfirmedChange()</code> . No AL call — was <code>InvokeExtensibilityMethod('OnRescheduleWorkOrder', ...)</code> on every drag before §3.1; the actual BC round-trip now only happens once, from the Confirm button.

Event	Behavior
onClick	Opens the (still-stubbed) Capacity Lookup for that event's skill/day — <code>openCapacityLookup(dayIdx, skill)</code> . Unaffected by §3.1 (a view action, not a schedule edit — see §3.1's closing note and the user's explicit confirmation to leave it as an immediate call).

Data source: `json.workOrderSequences[]` for row structure; per-cell values pull from `workOrderAssignmentState`, which reads `dayPlanningLines[]` filtered to `line.workOrderNo === inspectedWorkOrderNo` (§11/§13.3 for why this must be the normalized `workOrderNo`, not always the raw table field).

Bug fixed 2026-09-03

This section used to only pick up rows whose raw "Work Order No." table field equalled the inspected WO — but some of a Work Order's own genuine Day Planning Sequences (e.g. added via the Workorder Card's native "New sequence" button) can carry a blank value in that field. AL now scopes by `Job No. = WorkOrder."Project No."` AND `Job Task No. = WorkOrder."Project Task No."` instead.

Read this before deriving ANY total meant to match this section (§13.13)

The workday-offset positioning called out just above is not a cosmetic detail — it means this section's events, and the events array `renderWorkOrder()` builds internally (this._woEvents, §4.1), are the ONLY authoritative source for "what this section currently shows". Re-deriving a same-looking total from `dayPlanningLines[]`'s own stored Plan Date (or from `workOrderSequences[]` a second, independent way) produced two consecutive wrong results in the same session — see §13.13 for the full story before building anything new against this section.

NOT DHTMLX — hand-rolled HTML/CSS

7. Section 3 — Capacity vs Requested Daily Bars

Function: `renderCapacityBars(json) | Host: #cpo-capacity-bars`

This section is deliberately not a DHTMLX widget at all — it is plain generated HTML/CSS: for each visible day, a stacked "C" (Capacity: Assigned → Internal free → External free) column and a stacked "R" (Requested: Assigned → Unassigned-per-skill) column, plus Totals/Expand/Exp. to Task/Collapse buttons that control Section 4's tree open state (pure client-side, no AL round-trip). Column width is measured, not the literal `COLUMN_WIDTH` constant.

Data function	Reads	Produces
capParts(dayIndex)	<code>dayPlanningLines[]</code> (ALL work orders, unfiltered), <code>resources[].length × baseCapacity</code> , <code>externalFree[dayIndex]</code>	<code>{assigned, freeInt, freeExt}</code> — the "C" column. Company-wide on purpose.
dailyCapacityRequestData()	<code>capParts()</code> + <code>dayPlanningLines[]</code> EXCLUDING <code>line.workOrderNo === inspectedWO</code> (flipped 2026-09-15)	One row per visible day: <code>assigned, freeInt, freeExt, request, assignedRequest, unassignedBySkill{}</code> , shortage.

Click-to-relocate drives Section 2 — updated 2026-09-04

Clicking any non-weekend day cell in this section (not a double-click, which opens Capacity Lookup instead) calls `moveWorkOrderToDay(dayIndex)`: sets `woAnchor` directly to that day, re-renders Sections 1-3, and calls `markUnconfirmedChange()` — exactly like a Section 2 drag, just reached by a click here instead. As of §3.1, this is pure client-side with **no AL call at all** until the user clicks Confirm; before this session it fired `InvokeExtensibilityMethod('OnRescheduleWorkOrder', [dayIndex, ...])` on every click.

"R" (Requested) scoping FLIPPED 2026-09-15 — INCLUDE inspected-WO → EXCLUDE inspected-WO

Until this revision, `dailyCapacityRequestData()` kept ONLY the inspected Work Order's own lines (`line.workOrderNo === inspectedWO`), matching §11's original table. Explicit user request: the "Requested" side should instead show ALL company-wide day planning demand in the window MINUS the inspected Work Order's own lines (which Sections 1/2 already show) — i.e. the SAME direction Section 4's tree (§8) has used since 2026-09-03. Purpose: when the user drags a bar to relocate work to a different day, they need to see how much capacity OTHER work has already claimed that day, not just re-see their own WO's own demand. Live-verified against a real Work Order/Job Task: the old INCLUDE-only figure and the new EXCLUDE figure differed by roughly 60× on this company's real data (the inspected task's own demand is a small fraction of company-wide demand for the same window) — Section 3's new total now matches Section 4's own tree total exactly, confirmed via the live rendered DOM text on both this page and the Capacity Planning Dashboard tile (page 50724) for the same day. The "C" (Capacity) side is UNCHANGED — already company-wide/unfiltered by design (see §11).

"Show Data" drill-down fixed to match (same session)

Right-clicking a Request-column segment used to send the inspected Work Order's own Job No./Job Task No. as an INCLUDE filter to AL's day-planning-list drill-down (`CPO_OpenDayPlanningList/CPO_ShowCapacityBarSegment`, §10) whenever there was an inspected Work Order. Since the segment now represents "everything EXCEPT the inspected WO" after the flip above, sending that WO's own job/task as an INCLUDE filter would open the exact WRONG (opposite) record set. AL has no exclude-filter capability for this list today, so the fix is JS-only: the include-filter block was removed entirely — job/task now stay blank for a Request-segment drill-down on BOTH this page and the Dashboard (which already always sent blank). Known, accepted minor over-inclusion: "Show Data" on a Request segment now opens every matching skill+date Day Planning company-wide, a slight superset of what the segment visually represents (it also includes the excluded inspected WO's own lines) — documented inline as a deliberate trade-off, not a bug, revisit only if a real AL exclude-filter is ever added.

8. Section 4 — Skill → Job/Task → Sequence Tree

Function: `renderCentralTree(json)` | Host: `#cpo-central-tree` (wrapped by `#cpo-central-tree-wrap`, §13.5) | View: `centraltree`

The third real Scheduler instance, using `render: 'tree'` (the `treeline` render mode). Three levels: Skill (folder) → Job/Task "detail" row (folder) → Sequence (leaf). Leaf day-cells can render multiple side-by-side "chip" cells when several real Day Planning lines land on the same day/sequence; parent Skill/Task rows show one aggregated heat-map percentage cell instead.

createTimelineView option	Value	Note
render	'tree'	Only section using tree mode.
y_unit	<code>buildCentralSections()</code> output	Client-built Skill→Task→Sequence hierarchy from <code>groups[] + dayPlanningLines[]</code> .
dy / folder_dy	<code>ROW_HEIGHT = 32px</code> (both)	
section_autoheight	false	Intent: literal row height. In practice DHTMLX still fits total row height to its own init-target element's <code>clientHeight</code> — §13.5's bug.
dx / header width	<code>TREE_LABEL_WIDTH = 360px</code> (3-column: Skill/Job/Task)	Wider than sections 1-3's shared row label — a real 3-column grid header, own independent date row.
columns[0].template	<code>centralTreeLeftColumnHtml(o)</code>	Renders the 3-column Skill Job Task left panel per row.

Per-row/cell templates:

- `centraltree_cell_value` — Skill rows call `skillDaySummary()`; Job/Task "detail" rows call `taskDaySummary()` (both render a gradient-filled % heat cell); Sequence leaf rows call `sequenceDayCellHtml()` (renders the actual chip(s)). As of 2026-09-04 (§13.8), all three read from a precomputed `treeSummaryIndex()` instead of rescanning `dayPlanningLines[]` on every call.
- Chips are clickable/hoverable — `attachTreeChipTooltip()` binds one shared mouseover/click handler on the whole host div, reusing the same tooltip element Section 2 uses. Click raises `OnSequenceChipClick` (still an immediate AL call, see §3.1's closing note).

Bug fixed 2026-09-03 (scope)

`groups[]` (and therefore this whole tree) is built AL-side exclusively from every OTHER Work Order's demand in the visible window — the inspected Work Order's own data must never appear here. The exclusion criterion had to be corrected from the raw "Work Order No." field to a Job No./Job Task No. match against the inspected Work Order.

Bug fixed 2026-09-03 → follow-on fixed 2026-09-04 (V-scrollbar)

The ORIGINAL bug (§13.5): DHTMLX was auto-shrinking every row's rendered height to fit its own init-target element's reported height, so a tree with far more rows than fit never actually overflowed and no scrollbar ever appeared — fixed by splitting the host into an unconstrained inner div (DHTMLX's real render target) wrapped by an outer div carrying the MIN/MAX-clamped height + `overflow-y:auto`. **The FOLLOW-ON bug found 2026-09-04:** that height was only ever applied ONCE, at the initial render — collapsing/expanding rows afterward left the wrapper sized for whatever row count was visible last, so the scrollbar thumb kept spanning the fully-expanded range and a blank gap appeared below the actual (now shorter) row list. See §13.7 for the fix and live confirmation.

Performance fixed 2026-09-04

`skillDaySummary/taskDaySummary/sequenceDayLines` each used to rescan the ENTIRE `dayPlanningLines[]` array on every single rendered cell, making Expand/Collapse-all multi-second on real data even though nothing round-trips to BC. See §13.8 for the `treeSummaryIndex()` fix and live-measured before/after numbers, and §13.11 for why both skill and Job/Task folders now also start CLOSED on load rather than expanding that cost automatically.

9. The Shortage / Coverage Engine (Max-Flow)

Section 1's two rows and Section 3's implicit "shortage" figure are computed by a near-verbatim port of the reference prototype's own client-side algorithm — an Edmonds-Karp-style augmenting-path max-flow model (`maxFlowDay`), not a simple "sum capacity vs sum demand" formula.

Function	Purpose
<code>maxFlowDay(dayIndex, demandBySkill)</code>	Given one day's per-skill demand, computes the maximum achievable coverage across the (skill-scoped, capped) resource pool via flow conservation — achievable coverage is capped by demand, not supply.
<code>evaluateWO(dayIndex)</code>	"What if this Work Order's current schedule position is included?" — returns {coverage, addedShortage}, the source of Section 1's "Calculated conclusion" row.
<code>currentPositionShortage() / currentPositionSkillShortage()</code>	Before/after delta: <code>maxFlowDay</code> WITHOUT this WO's demand vs WITH it, at its current <code>woAnchor</code> position — the source of Section 1's "Current position shortage" row and Section 2's per-row shortage badges.

Why the algorithm was reversed back in (2026-09-02 decision)

An earlier, simplified "sum team capacity minus other-WO-assigned-hours" formula had a real bug — a large company-wide resource pool produced coverage numbers over 5,700%. The max-flow model doesn't have this failure mode because achievable throughput is fundamentally bounded by demand, never by an oversized supply pool.

Performance caveat: max-flow is expensive (roughly $O(\text{resources}^2)$ per day) and is called from BOTH Section 1's per-cell templates AND the shortage functions' own per-day loops, with no memoization in the reference. This port adds per-render-pass caching (`this._evalWOCache`, `this._currentPositionShortageArr`, cleared on every `applyPlanningData()` / `resetAnchorDependentCaches()`) and AL caps the resource pool at 15 per skill (`CPO_MaxResourcesPerSkill`) — both are deliberate, documented performance trade-offs, not data-completeness bugs. This caching is unaffected by the 2026-09-04 redesign: `resetAnchorDependentCaches()` is still called on every local reschedule move, before or after §3.1 — only the AL call at the end of that path changed.

10. AL-Side Implementation Details

codeunit 50604 "DHX Data Handler" — CPO_ region, key procedures

Procedure	Purpose
CPO_BuildPlanningDataJson_Paged(WorkOrderNo, NumberOfDays, MaxOtherLines, out RemainingGroupKeys): Text	The one entry point — assembles the entire payload in the 3-pass + pagination shape described in §12.
CPO_BuildDayPlanningLineObj(DayPlanning, InspectedWorkOrderNo, InspectedJobNo, InspectedJobTaskNo): JsonObject	One <code>dayPlanningLines[]</code> entry. Normalizes <code>workOrderNo</code> to the inspected WO's own No. whenever the line's Job No./Job Task No. match it.
CPO_BuildSkillsArray / CPO_BuildResourcesArray / CPO_BuildExternalFreeArray / CPO_BuildGroupsArray	Build <code>skills[]</code> / <code>resources[]</code> / <code>externalFree[]</code> / <code>groups[]</code> respectively — §4 for each one's exact scope.
CPO_BuildWorkOrderSequencesArray	Builds <code>workOrderSequences[]</code> from the Pass-1 dedup lists.
CPO_ComputeWorkdayOffset / CPO_IndexOfText / CPO_BuildDateRangeArray / CPO_FormatTimeHHMM	Small shared helpers (offset math, list lookup, date range, time formatting).
CPO_MaxResourcesPerSkill(): Integer	Returns 15 — the max-flow performance cap (§9).

page 50722 "Capacity Planning Overview"

- No `SourceTable` — a bodiless Card page holding exactly one `usercontrol`.
- `Caption = ""`; — blank on purpose (the JS renders its own title bar).
- `SetWorkOrderNo(pWorkOrderNo)` — filter-setter-before-Run(), called by the launching page before opening.
- `DaysToShow` global var, default 30 (`DefaultDaysToShow()`) — page-level state; the reschedule flow's pending "unconfirmed change" state (2026-09-04) lives entirely client-side, NOT mirrored into any AL variable.
- `RefreshData()` — single shared rebuild-and-push routine called by `ControlReady` and `OnDaysToShowChanged`.
- **`OnRescheduleWorkOrder(DayShift, PayloadJsonTxt)`** — unchanged signature; 2026-09-04 removed its `Message('Reschedule stub: shift=%1', DayShift)` body (§13.10), now a true silent stub matching

OnRequestCapacityLookup/OnSequenceChipClick. Only ever invoked from the Confirm button now (§3.1), not per drag/click.

Pag50662.WorkorderCard.al — launching action

```
action(CapacityPlanningOverviewAct)
{
  trigger OnAction()
  var CPO: Page "Capacity Planning Overview";
  begin
    CPO.SetWorkOrderNo(Rec."Work Order No.");
    CPO.Run();           // plain top-level navigation - see 13.6
  end;
}
```

11. Cross-Work-Order Scoping (2026-09-03 Correction)

Explicit product decision, stated by the user in these exact terms: "section 3 for DWO0008 but section 4 for all day planning data in days time frame except DWO0008", further clarified as scoped "as long as in same time frame of days" and explicitly including "other WO in other job" (i.e. truly company-wide, not scoped to the inspected WO's own parent Job).

Section	Scope	Rationale
Section 1 (stats)	Baseline EXCLUDES inspected WO (reframed 2026-09-15)	§9's excludingWOFlow() reads the "everyone else, minus this WO" baseline DIRECTLY (no longer a before/after delta against it) — same exclude direction as Section 3/4 below, just consumed differently.
Section 2 (own scheduler)	Inspected WO only	"This WO's own demand." Unchanged.
Section 3 — "R" (Requested)	Company-wide, EXCLUDING inspected WO (flipped 2026-09-15)	Was "inspected WO only" through 2026-09-14 — flipped to match Section 4's own direction so a drag-relocate shows genuine competing demand from everyone else that day. See §7.
Section 3 — "C" (Capacity)	Company-wide (unfiltered)	"Everything else going on in this window, for contention/context." Unchanged.
Section 4 (tree)	Every OTHER Work Order, ANY Job, same date window	Intentionally the opposite of the earlier, narrower behavior ("this WO's own tree") — explicit and permanent. Unchanged.

Still true and unchanged by the 2026-09-04 confirm-gated redesign: nothing about §3.1 alters WHICH data each section shows, only WHEN (if ever) a reschedule action tells BC about a change — Section 4's scope in particular is completely untouched, since it was already never re-rendered by any reschedule path.

Table revised 2026-09-15

The Section 1 and Section 3 — "R" rows above were both revised this session (§5, §7, §13.12) — Section 3's own direction is a genuine flip (was include-only, now exclude-only); Section 1's "own position" framing is retired along with the before/after delta engine it described, replaced by a direct read of the same excluding-this-WO baseline Section 3/4 already use. Section 2 and Section 4 are unchanged by this revision.

12. Performance — Page Background Task Pagination

Added 2026-09-03, on explicit user request. As Section 4's cross-Work-Order scope (§11) made the add-in query every other Work Order's Day Planning demand company-wide for the visible window, real companies with a large Day Planning volume for that period made JSON generation (AL) and DHTMLX ingest (JS parse + tree/model build + render) slow enough to notice on first paint. The fix ports this repository's existing Page Background Task pattern — already used by `request_assignment` (codeunit 50721), `projectschedule` (codeunit 50720), and `ganttdemo2` (codeunit 50713) — into CPO, following the exact same shape.

12.1 The technique (Business Central Page Background Task)

API	Role
<code>CurrPage.EnqueueBackgroundTask(TaskId, Codeunit, Parameters, TimeoutMs, ErrorLevel)</code>	Starts a child session running the given codeunit's OnRun trigger asynchronously, off the interactive request path. Returns immediately.
<code>Page.GetBackgroundParameters(): Dictionary of [Text, Text]</code>	Read inside the child codeunit's OnRun to retrieve the parameters passed at enqueue time.
<code>Page.SetBackgroundTaskResult(Dictionary of [Text, Text])</code>	Called at the end of the child codeunit's OnRun to hand results back to the parent page.
<code>trigger OnPageBackgroundTaskCompleted(TaskId, Results)</code>	Fires on the parent page when the child session finishes. Cannot safely call the control add-in from here — must stash the result into a plain AL variable instead.
<code>trigger OnPageBackgroundTaskError(TaskId, ErrorCode, ErrorText, ErrorCallStack, var IsHandled)</code>	Fires on failure/timeout. Surfaced via a Notification, not a blocking error.

Why results can't be pushed to the control add-in directly from `OnPageBackgroundTaskCompleted`: confirmed live across four independent add-ins in this repository — BC Server rejects a control add-in callback issued directly from that trigger. The workaround: stash the result into a plain page variable, then let a JS-initiated poll (a normal synchronous trigger call, safe to answer with a control add-in callback) pick it up and deliver it.

12.2 CPO's architecture (mirrors `request_assignment`'s board exactly)

```

page 50722 RefreshData()
    CPO_BuildPlanningDataJson_Paged(WorkOrderNo, DaysToShow, MaxOtherLines=50, out RemainingGroupKeys)
    CurrPage.DhxCpo.SetPlanningData(json) // first paint: fast
    EnqueueOtherWorkOrderDataBackgroundTask(RemainingGroupKeys)
    CurrPage.EnqueueBackgroundTask(.., Codeunit::"CPO BG Other WO Data", params, 30000, ..)
    CurrPage.DhxCpo.NotifyOtherWorkOrderDataTaskPending() // JS starts polling + shows #cpo-bg-loading (3.2)

(async, off the interactive request path)
codeunit "CPO BG Other WO Data".OnRun()
    CPO_BuildOtherWorkOrderLinesJson_ForKeys(WorkOrderNo, StartDate, EndDate, RemainingGroupKeys)
    Page.SetBackgroundTaskResult({ otherWorkOrderDataJson: json })

trigger OnPageBackgroundTaskCompleted(TaskId, Results)
    stash Results into PendingOtherWorkOrderDataJson (plain var - no control add-in call here)

trigger OnPollOtherWorkOrderDataResult() // JS-initiated, polled every 500ms
    CurrPage.DhxCpo.AppendOtherWorkOrderData(PendingOtherWorkOrderDataJson)
    CurrPage.DhxCpo.StopOtherWorkOrderDataPolling() // also hides #cpo-bg-loading (3.2)

```

12.3 The atomic pagination unit — whole Skill+Job No.+Job Task No. groups, never split

Exactly like request_assignment's sequenceKey pagination, CPO never splits a rendering unit across the synchronous first page and the background remainder. The unit here is a whole group — Skill+Job No.+Job Task No., the same combination Section 4's tree groups by.

Pass	Cost	Scope	Produces
CPO_ScanOtherWorkOrderGroups (Pass 3a)	Cheap — key fields only, no per-line JSON	ALWAYS full — every group, never paginated	Complete skills[]/resources[]/externalFree[]/groups[] + the group order/line-count needed to decide the page cutoff
CPO_BuildOtherWorkOrderLinesForGroups (Pass 3b)	Expensive — lookups, time-to-decimal, full per-line JSON	Paginated — only whichever whole groups are in WantedGroupKeys	dayPlanningLines[] entries — the expensive part, and the only part actually deferred

Practical effect: Section 4's tree skeleton renders complete on first paint, even over a huge dataset. Only the Sequence-level leaf chips for groups past the first 50 backfill progressively once the background task delivers them, typically within the same second or two on this demo dataset (confirmed live: 1,388 distinct groups across 8 skills, 5,042 total Day Planning lines merged successfully with zero console errors).

12.4 Why Section 1/2/3 are treated differently

Section	Paginated?	Reason
Section 2 (inspected WO's own scheduler)	No — Pass 1/2 unchanged	Bounded to one Work Order's own lines — never the actual bottleneck.

Section	Paginated?	Reason
Section 1 (stats) / Section 3 (capacity bars)	Indirectly — provisional until backfill completes	Computed from the full company-wide dayPlanningLines[]. Correct for whatever data has arrived so far, self-corrects once the remainder appends.
Section 4 (tree)	Yes — the actual target	Skeleton always complete; chip-level detail backfills per §12.3.

This "provisional until backfill completes" row is exactly §13.12's finding

A verification pass on this add-in (2026-09-15) treated a Section-3-vs-Section-4 self-consistency check as proof the numbers were trustworthy, without checking whether the background load in THIS table row had actually finished — both sections read the identical (still-partial) dayPlanningLines[] array, so they agreed with each other while both under-counting the true company-wide total by roughly 62×. See §13.12 for the full story and the concrete check (poll #cpo-bg-loading) that would have caught it.

12.5 The JS-side append — why it's more than a tree rebuild

CPO's shortage/coverage engine (§9) and Section 3's capacity bars are computed from the full dayPlanningLines[] set, so appendOtherWorkOrderData(rawLines) must: (1) push the new lines into this.db.dayPlanningLines; (2) invalidate every derived cache (including, as of 2026-09-04, this._treeSummaryIndex — §13.8); (3) re-render Sections 1, 3, and 4 and re-bind the shared scrollbar. Section 2 is not re-rendered — its own source data was never paginated.

12.6 New/changed objects (2026-09-03)

Object	Change
codeunit 50722 "CPO BG Other WO Data"	New. Background task target — modeled directly on codeunit 50721.
codeunit 50604: CPO_BuildPlanningDataJson_Paged	New. Paged variant of CPO_BuildPlanningDataJson.
codeunit 50604: CPO_ScanOtherWorkOrderGroups	New. Pass 3a — the cheap, always-full pre-scan.
codeunit 50604: CPO_BuildOtherWorkOrderLinesForGroups	New. Pass 3b.
codeunit 50604: CPO_BuildOtherWorkOrderLinesJson_ForKeys	New. Background-task companion.
page 50722	RefreshData() now calls the paged builder + EnqueueOtherWorkOrderDataBackgroundTask; new background-task triggers.
Controladdin	New procedures/events for the pagination poll.
wrapper.js	New poll timer + window.AppendOtherWorkOrderData.
capacityPlanningOverview.js	New appendOtherWorkOrderData(rawLines) method.

13. Known Pitfalls, Bugs Found & Fixes Applied (session log)

Renumbered 2026-09-04 — this section's entries were previously mislabelled 12.1–12.6 (reusing §12's numbers, a copy-paste artifact of the original document); they are correctly 13.1–13.6 below, with that session's own findings added as 13.7–13.11, and this

revision's findings added as 13.12–13.13.

13.1 Unterminated CSS comment silently disabled the top bar

style.css had a comment inside #cpo-root {} that opened (/*) but never closed before that block's own }, so the first real */ the parser then found was ~30 lines later, silently swallowing .cpo-top-bar's and .cpo-top-title's entire rule bodies as dead comment text. Symptom: the title and "Days to show" input stacked on two plain, unstyled rows instead of one styled row. Fix: add the missing */ immediately after the intended one-paragraph comment.

Lesson

An unterminated CSS comment doesn't error — it silently consumes everything until the next literal */ anywhere later in the file. Always grep for balanced /* */ pairs when a rule that "should" apply visibly doesn't.

13.2 RunModal() vs Run() — native BC chrome

Opening the page via RunModal() always renders BC's own dialog chrome (bold title bar + command bar + Close button) regardless of the page's own Caption or field content. Switching to Run() (plain top-level navigation, the same pattern page 50710 "DHX Request Assignment Board" already used) replaces it with just a back arrow, matching the reference prototype's own minimal chrome.

13.3 "Work Order No." field vs Job No./Job Task No. scoping

See §6/§8/§11. The native Workorder Card's own "Day Planning Sequence" part filters Day Planning by Job No./Job Task No., never by the "Work Order No." field. Some genuine Day Planning Sequences can have that field blank. Any AL query that scopes "this Work Order's own data" by the "Work Order No." field alone will silently miss such rows — and symmetrically, any exclusion keyed the same way will wrongly include them as if they belonged to someone else.

13.4 Flexbox default shrink crushed the shared scrollbar

#cpo-root is a fixed-height display:flex; flex-direction:column container. #cpo-shared-scroll (declared height:16px) had no flex-shrink override, so whenever the other sections' combined natural height exceeded #cpo-root's own bounded height, the flex column's default flex-shrink:1 proportionally squashed every child — measured live: rendered height collapsed to 1px. Fix: flex: 0 0 16px; (no shrink, no grow, fixed 16px basis).

13.5 DHTMLX auto-fits row height to its container — scrollbars never appear (original fix)

Even with dy/folder_dy fixed and section_autoheight:false, DHTMLX Scheduler reads its own init-target element's clientHeight at render time and fits all row content to exactly that height (shrinking rows) rather than rendering at the literal configured height and overflowing. Fix (Sections 2 and 4): split the single host div into two nested divs — DHTMLX initializes into an inner div left height-unconstrained, while a new outer wrapper div carries the MIN/MAX-clamped height and overflow-y:auto that actually does the visual clipping/scrolling. See §13.7 for the 2026-09-04 follow-on to this fix.

13.6 Concurrent publish/build collisions

Running multiple al_publish operations from parallel sessions/subagents against the same sandbox environment can produce a spurious CompilationFailed even when al_getdiagnostics shows zero real errors — a file-lock/build collision, not a code defect. Resolved by waiting for the other publish to finish and retrying.

13.7 Section 4 scrollbar drifted from the actual row composition after collapse/expand

New, 2026-09-04. §13.5's fix solved the INITIAL scrollbar bug, but applyCentralTreeHeight was only ever called once, at the initial renderCentralTree pass — collapsing/expanding rows afterward (via setTreeOpenState's Expand/Collapse-all buttons, or a single row's own native fold arrow) never re-ran it, so the wrapper's clip height stayed sized for whatever row count was visible at the LAST full render. Symptom (reported by the user with a screenshot): collapse all rows, and the scrollbar thumb still spans the fully-expanded range, leaving a blank scrollable gap below the now-much-shorter row list.

Fix

bindCentralTreeHeightSync listens for DHTMLX's onOptionsLoad event (fired both by setTreeOpenState and by a single row's own native toggle) and re-applies applyCentralTreeHeight against the LIVE tree state every time. setTreeOpenState additionally pre-applies the height BEFORE firing onOptionsLoad, so DHTMLX's own native redraw already reads the corrected height on its one pass — no forced second redraw needed (an earlier attempt added an explicit setCurrentView() call here on the theory that DHTMLX's native handler might read a stale height otherwise; §13.8 explains why that call was removed again).

Confirmed live

Collapse-all shrank the wrapper's scrollHeight from 90,476px to 300px (the MIN_TREE_HEIGHT clamp); a single row's own toggle correctly produced an intermediate 24,012px; Expand-all scrolled cleanly to the true last row with no blank gap in all three cases.

13.8 $O(\text{rows} \times \text{days} \times \text{dayPlanningLines})$ per-cell rescans made Expand/Collapse-all multi-second

New, 2026-09-04 — raised by the user as "as Data already in JS, why take long time during expands and collapse?". skillDaySummary/taskDaySummary/sequenceDayLines each did a full .forEach/.filter scan over the ENTIRE dayPlanningLines array (this WO's lines plus every other WO's — §11) — and DHTMLX calls each of these once per RENDERED CELL (every visible row $\times$ every visible day) during its native tree redraw. Real cost: $O(\text{rows} \times \text{days} \times \text{dayPlanningLines.length})$ — genuine, user-visible latency purely on the client, unrelated to any AL round-trip.

A wrong first diagnosis, corrected by profiling

The §13.7 fix's own earlier draft added an explicit setCurrentView() call on the theory that it was needed for correctness. When THIS bug was investigated, profiling with that call instrumented (not yet removed) showed it consuming ~5.4s of a ~5.4s total cost — but removing it entirely and re-profiling showed the SAME ~6.3s cost still fell inside DHTMLX's own native redraw call, proving the extra call was never the real bottleneck. Lesson (explicitly worth keeping): attribute a regression to a specific recent change only after profiling the counterfactual, not by inspection or recency alone.

Fix

treeSummaryIndex() groups dayPlanningLines ONCE per data load ($O(\text{dayPlanningLines.length})$ total, invalidated on the same applyPlanningData/appendOtherWorkOrderData calls that already reset the other derived caches, §9) into skill+day / skill+job+task+day / skill+job+task+sequence+day lookup maps. The three summary functions became $O(1)$ map lookups.

Confirmed live

On DWO0006's real data, Expand-all dropped from ~5.4-6.3s to ~1.7s ($\approx 3.5\times$), with identical output (same hour totals, no squished/misaligned rows, correct scrollbar behavior per §13.7).

13.9 InvokeExtensibilityMethod is fire-and-forget from JS's own perspective

New finding, 2026-09-04 — discovered while investigating §13.7/§13.8 and directly informing the §3.1 redesign and §3.2's indicator design. Confirmed live by wrapping the call with performance.now() timing: the JS call itself returns in well under 1ms regardless of what the AL trigger it targets actually does — even a genuinely slow AL round-trip (a "Days to show" reload rebuilding company-wide data took ~82s wall-clock in one measurement) does not block the calling JS thread. The real response, if any, always arrives later through a SEPARATE AL-to-JS call (SetPlanningData, AppendOtherWorkOrderData, LoadCapacityLookup), never as a return value from the invoke itself.

Practical effect on this add-in's own design

showLoading() called immediately before InvokeExtensibilityMethod always gets a real paint for free, since the call yields control back to the browser well before AL finishes — no requestAnimationFrame deferral trick needed there (openCapacityLookup, the Section 4 chip click, confirmChanges all rely on this). That trick (runBusy, §3.2) is needed ONLY around genuinely synchronous CPU-bound client work with no natural browser yield point — DHTMLX's own tree redraw (§13.8), specifically.

13.10 Reschedule stub feedback: blocking dialog → JS notice → removed entirely for a Confirm button

New, 2026-09-04. OnRescheduleWorkOrder's stub originally proved its wiring fired via a native Message('Reschedule stub: shift=%1', DayShift) dialog. That was reasonable when reschedule was a rare, deliberate action — but at the time every single drag/click-to-relocate fired it, interrupting the user with a mandatory OK click on every intermediate step (screenshot evidence: a dialog stacked directly over the schedule mid-drag).

Iteration 1 — non-blocking JS notice

Removed the AL Message() call, replaced with a small non-blocking JS-rendered notice (a bordered text box in Section 3's fixed left column, auto-hiding after 4s) showing the same "Reschedule stub: shift=N" text, computed entirely client-side (the shift value was already known in JS before ever invoking AL).

Iteration 2 (final) — removed entirely

Once reschedule became fully local-simulation + Confirm-gated (§3.1), the per-action notice's whole reason for existing disappeared too — there is no longer a per-action AL call to "prove fired". Removed; the Confirm button's own enabled/pending visual state (.cpo-confirm-btn-pending) is what now signals "you have an unconfirmed move" instead.

13.11 Expand-all still felt slow after §13.8 — three dead ends before the real fix

New, 2026-09-04 — even after treeSummaryIndex (§13.8) cut Expand-all to ~1.7-2.1s, the user kept pushing on "it must be faster, data is already in DHTMLX" (fair — 1.7s still reads as sluggish for a purely local operation). Three approaches were tried and profiled live before landing on the one that actually worked.

Dead end 1 — DHTMLX Scheduler's own smart_rendering option

Scheduler's minified bundle ships a viewport-based lazy-render path (_timeline_smart_render.getViewPort). Enabling smart_rendering: true on the centraltree view was a natural next try — measured live, it made Expand-all SLOWER (~8.1s, not faster). Reverted. Likely not effective for render:'tree' mode's folder hierarchy the way it is for a flat timeline.

Dead end 2 — swap Section 4 to the DHTMLX Gantt library

The Gantt add-in's (ganttdemo2) own collapse/expand (gantt.eachTask(t=>t.\$open=open); gantt.render()) is fast because DHTMLX Gantt virtualizes rows natively. Investigated as a full swap; found two real obstacles first: Gantt's LEFT grid/tree panel does not scroll horizontally on its own (only its chart/timeline pane does), and that chart pane draws task BARS positioned by date range, not arbitrary per-cell HTML the way Scheduler's cell_template does — Section 4's heat-cells/chips would need to become day-wide bars to fit that model. Surfaced to the user as a scoped decision before any code was written; the user chose to stay on Scheduler rather than take on that rewrite.

Dead end 3 — "copy Task Scheduler's mechanism"

The user pointed at Task Scheduler (projectschedule) as an example of fast expand/collapse still on DHTMLX Scheduler. Reading its actual ToggleCollapseExpandAllSections in projectschedule/wrapper.js showed it is BYTE-FOR-BYTE the same technique CPO's own setTreeOpenState already uses (same y_unit_original mutation, same _getArrayToDisplay, same onOptionsLoad call) — CPO's own code comment already says it was ported from that exact function. There was no different mechanism to copy. The real gap is scale: Task Scheduler's tree is Job→Job Task (scoped, few rows); CPO's Section 4 fully expanded to Sequence level, company-wide (§11), is 2,831 rows — DHTMLX's native redraw cost after onOptionsLoad scales with row count regardless of which code toggles the open flags.

Fix — start collapsed, pay the cost only when asked

buildCentralSections() now sets open: false for BOTH the skill and the Job/Task "detail" level on every render pass, regardless of AL's own groups[].expanded flag (which drove the skill level's initial state before). Expand-all's ~1.7-2.1s is real, unavoidable-at-that-scale DOM-construction cost for 2,831 rows — the fix is not making that number smaller, it is making sure the user only ever pays it when they actually click Expand, not on every page open.

Confirmed live

Fresh page load: all 8 skill rows render collapsed instantly (wrapper clientHeight/scrollHeight both 300px, the MIN_TREE_HEIGHT clamp - no scrollbar needed at all). Expand-all afterward: scrollHeight grows to 90,636px in ~2.16s (the §13.8-era cost, now opt-in only), and scrolling to the bottom lands exactly at the true last row with no blank gap - confirming §13.7's scrollbar-sync fix still holds with the new default state.

13.12 Trusting a total before the background load (§12) actually finishes — a 62× under-count

New, 2026-09-15. The Section 3 exclude-flip (§7) was first verified live by summing dailyCapacityRequestData()'s own output over the visible 30-day window and comparing it against Section 4's own tree total (§8) — both read 376h, a perfect match, reported as confirmed. The coordinator pushed back: a real live figure the user had separately seen on the Capacity Planning Dashboard tile for a single day (3,304h) was far larger than the ENTIRE 30-day total just reported. Investigating found the real cause: this add-in's "other Work Order" data loads via the §12 Page Background Task, and the original check captured db.dayPlanningLines at only 107 lines — the FIRST synchronous page. The true, fully-loaded company-wide total was 6,628 lines, a 62× under-count. Section 3 and Section 4 agreed with EACH OTHER throughout, because both read the exact same (still partial) array — a self-consistency check that is structurally blind to under-loading.

Why the self-consistency check passed anyway

Section 3 and Section 4 are two different VIEWS of the identical db.dayPlanningLines array. Comparing them against each other only proves they agree on whatever data currently exists client-side — it proves nothing about whether that data is complete. The only way to catch this is to check completeness directly, or compare against a source that does NOT share the same (possibly stale) input.

Fix — the concrete check for any future verification pass

Poll document.getElementById('cpo-bg-loading').style.display (inline style, set by showBackgroundLoading()/hideBackgroundLoading() in capacityPlanningOverview.js) before trusting any Section 3/4 total on page 50722 specifically (page 50724/Dashboard has no such background task — its one query is synchronous and complete on load). 'none' means the load is done, or was never needed (everything fit on the first synchronous page); anything else means db.dayPlanningLines — and everything derived from it — is still incomplete.

Confirmed live once actually complete

After the background load finished (6,628 lines), Section 3's Wednesday total read 3,304h, Section 4's tree read the same 3,304h, the inspected Work Order's own demand that day was 0h, and $3,304 + 0 = 3,304$ = the Dashboard's own real figure for the same day — confirmed via the live RENDERED DOM text on both pages, not just internal JS state.

A separate, reproduced-but-unresolved reliability gap, worth flagging rather than fixing here: the background load itself was observed to be genuinely slow (well over a minute in one run) and, in a separate run against the identical Work Order/page, appeared to stall indefinitely with a real console error logged once (`OnPollOtherWorkOrderDataResult poll failed: undefined, wrapper.js:119`) and no further progress observed within a several-minute patience budget. Reproduced twice, same day, no AL/JS file in the §12 pagination path was touched by this session — an existing, pre-session intermittent gap in the background-task plumbing, not a regression. A future session investigating this add-in's performance/reliability should expect to need real patience (or several fresh page-open attempts) before the full company-wide dataset is actually available to test against.

13.13 Section 2's chips are NOT a render of `dayPlanningLines[]`'s stored Plan Date — two consecutive wrong fixes came from assuming they were

New, 2026-09-15 — read this before touching the title-bar audit box (§4.1), Section 2 (§6), or anything else that needs to match "what Section 2 visibly shows" ever again. While building §4.1's audit-totals box, TWO consecutive fixes were shipped, both wrong, both because of the same false assumption: that Section 2's rendered chips are a direct visualization of `db.dayPlanningLines[]`'s own stored "Plan Date" field. They are not.

What Section 2 actually renders from

`renderWorkOrder()` (§6) builds its events array from `db.workOrderSequences[].workdays[]`, mapped through `getOccurrenceDayIndex(seq, wd)` — which places every occurrence RELATIVE TO THE CURRENTLY VISIBLE WINDOW'S OWN START (`this.dates[0]`), completely independent of whatever real calendar date is stored on the underlying Day Planning record. Each event's hours come from `seq.hoursByWorkday[wd]` (defaulting to 8 if AL sent none), not from re-reading `dayPlanningLines[].requestedHours` for that date. This is a deliberate, SEPARATE client-side "reschedule preview" simulation — matches this file's own `drag/moveWorkOrderToDay` design (§3.1): no AL round-trip, no re-fetch, purely a local repositioning of "day N of this Work Order's own schedule" starting from wherever the window happens to begin right now.

Fix attempt 1 (documented in an earlier draft of this section, since corrected): summed `db.dayPlanningLines[]` filtered only by `workOrderNo === woNo`, no date check at all. Independently re-verified against a direct OData query on the real Day Planning table and found numerically correct as an ALL-TIME total — but Section 2 doesn't show an all-time total, so a live screenshot showing many visible chips against a small non-zero audit total looked "impossible".

Fix attempt 2: added a `dplDayIndex(line)-inside-the-visible-window` check on the SAME `db.dayPlanningLines[]` array, reasoning that the box should be window-scoped like Section 3. This produced a plausible-looking "0/0" for one real test case (the Work Order's only real-dated demand happened to fall just before that day's visible window) — but was flagged wrong again immediately: a DIFFERENT window (15 Sep–05 Oct) showed many chips clearly INSIDE the visible range while the box still read 0/0, because `dayPlanningLines[]`'s literal dates and `workOrderSequences[]`'s window-relative positions are simply two different concepts that do not need to agree, and usually won't, for any Work Order whose real schedule doesn't happen to start exactly on "today".

The actual fix

renderWorkOrder() now stores its own freshly-built events array on the instance (this._woEvents = events) immediately before handing it to DHTMLX. A new shared method, computeWorkOrderTotals(), sums requestedHours/assignedHours over THAT array, and renderTitleAuditBox() reads it to update the title box — called from the end of renderWorkOrder() itself, so it refreshes on every reposition, not just the initial load. There is now exactly ONE array Section 2 and the title box both read; they are structurally incapable of disagreeing, since neither re-derives the same-looking number a different way. See §4.1 for the full implementation.

The generalizable lesson, worth restating a third time since it took three attempts to land: in this add-in, "what date is this occurrence shown on" and "what date is this real Day Planning record stored under" are NOT the same question, and a feature that needs to match Section 2's own on-screen behavior must ask Section 2's own question (workOrderSequences[]/getOccurrenceDayIndex, or better, Section 2's own already-built events array) — never assume dayPlanningLines[]'s literal Plan Date is an acceptable proxy for it, no matter how reasonable that assumption feels, and no matter how convincing an independent ground-truth check of the WRONG array's own correctness looks.

14. Appendix — File Reference

Path (relative to repo root)	Contents
srcldhx\capacity_planning_overview\DHXCapacityPlanningOverviewAddin.ControlAddin.al	Controladdin declaration
srcldhx\capacity_planning_overview\page_50722_CapacityPlanningOverview.al	Host page 50722
srcldhx\capacity_planning_overview\startupScript.js	BOOT() call
srcldhx\capacity_planning_overview\wrapper.js	BOOT/binding shell + loading-overlay safety timer (§3.2)
srcldhx\capacity_planning_overview\capacityPlanningOverview.js	The single JS component — all 4 sections + shortage engine (excludingWOFlow, §9, replacing evaluateWO/currentPositionShortage) + Confirm-gated reschedule state machine (§3.1) + processing indicators (§3.2) + treeSummaryIndex (§13.8) + title-bar audit-totals box (§4.1)
srcldhx\capacity_planning_overview\style.css	All layout/heat-map/chip/scrollbar/spinner/Confirm-button CSS
src\codeunit\codeunit_50604_DHXDataHandler.al	Shared data-handler codeunit — CPO_ region builds the payload
srcldhx\capacity_planning_overview\codeunit_50722_CPOBGOtherWorkOrderData.al	Page Background Task target — §12
src\page\Pag50662.WorkorderCard.al	Launching action CapacityPlanningOverviewAct
srcldhx\dhtmlx\scheduler.js / .css	Shared vendor Scheduler library (root-level, reused by every DHX add-in)
srcldhx\suite.js / .css	Shared vendor Suite library (declared but not currently used by this add-in's own modal, which is stubbed)
C:\Users\Ahmad\OneDrive\PROJECT\KLAAS\CPO_v131\DHTMLXtempv112-app.js	Reference prototype this add-in is ported from (external to this repo)

End of document.